

INDEX

NAME: Neha Ramich STD.: _____ SEC.: _____ ROLL NO.: _____ SUB.: _____

S. No.	Date	Title	Page No.	Teacher's Sign / Remarks
1.	23/2/26	Lab 0		
2.	6/3/26	Lab 1: FCFS, SJF, SRTF	(10)	SK
3.	13/3/26	Lab 2: Priority Scheduling	(10)	SK 9/3
				13/3
4.	28/3/26	LAB 3: Round Robin Multi level Queue.	(10)	SK
				27/3
5	10/4/26	LAB 4: Real Time CPU scheduling	(10)	SK
				10/4
6	24/4/26	Lab 5: Bounded Buffer Dining Philosopher	(9)	SK
				24/4
7.	8/5/26	LAB 7: Banker's algo Deadlock detection	(10)	SK
				8/5
8.	15/5/26	Lab 8: Contiguous Memory allocation.	(10)	SK
				15/5

LAB - a

① Second smallest element in an array :

```
#include <stdio.h>
#include <limits.h>

int main() {
    int n, i;
    int arr[100];
    printf("Enter no. of elements: ");
    scanf("%d", &n);
    printf("Enter elements: ");
    for (i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    int min = INT_MAX;
    int smin = INT_MAX;
    for (i = 0; i < n; i++) {
        if (arr[i] < min) {
            smin = min;
            min = arr[i];
        }
        else if (arr[i] < smin && arr[i] > min) {
            smin = arr[i];
        }
    }

    printf("The second smallest element is %d", smin);
    return 0;
}
```

O/P:

Enter no. of elements: 3

Enter elements: 3

4

7

The second smallest element is 4.

② Second largest element in array:

```
#include <stdio.h>
```

```
#include <limits.h>
```

```
int main() {
```

```
    int n, i;
```

```
    int arr[100];
```

```
    printf("Enter no. of elements: ");
```

```
    scanf("%d", &n);
```

```
    printf("Enter elements: ");
```

```
    for (i = 0; i < n; i++) {
```

```
        scanf("%d", &arr[i]);
```

```
    }
```

```
    int max = INT_MIN;
```

```
    int smax = INT_MIN;
```

```
    for (i = 0; i < n; i++) {
```

```
        if (arr[i] > max) {
```

```
            smax = max;
```

```
            max = arr[i];
```

```
else if (arr[i] > smax && arr[i] < max) {  
    smax = arr[i];  
}
```

```
}
```

```
printf ("The sec largest element is %d", smax);  
return 0;
```

```
}
```

O/P:

Enter no. of elements : 3

Enter elements : 3

4

7

The second largest element is 4.

~~gk~~

③ Sum of rows & columns in a matrix.

#include <stdio.h>

int main() {

int n, i, j;

int arr[100][100];

printf("Enter dim of matrix: ");

scanf("%d", &n);

printf("Enter elements: ");

for (i = 0; i < n; i++) {

for (j = 0; j < n; j++) {

scanf("%d", &arr[i][j]);

}

for (i = 0; i < n; i++) {

int sum = 0;

for (j = 0; j < n; j++) {

sum += arr[i][j]; }

printf("Sum of row %d = %d\n", i+1, sum);

}

```
for (j=0; j<n; j++) {
```

```
    int sum = 0;
```

```
    for (i=0; i<n; i++) {
```

```
        sum += arr[i][j];
```

```
    printf ("Sum of column %d is %d", j+1, sum);
```

```
}
```

```
}
```

O/P :

Enter dim of matrix : 3

Enter elements :

2

3

4

5

6

7

8

9

Sum of row 1 :

Sum of row 2 :

Sum of row 3 :

Sum of column 1 :

Sum of column 2 :

Sum of column 3 :


```

3) ④ Sum of all elements of a matrix
#include <stdio.h>

int main() {
    int n, i, j;
    int a[100][100];
    printf("Enter dim of matrix: ");
    scanf("%d", &n);
    printf("Enter elements: ");
    for (i = 0; i < n; i++) {
        for (j = 0; j < n; j++) {
            scanf("%d", &a[i][j]);
        }
    }
}

```

```

int sum = 0;
for (i = 0; i < n; i++) {
    for (j = 0; j < n; j++) {
        if (i == j) {
            sum += a[i][i];
        }
    }
}

```

```

printf("Sum = %d", sum);
return 0;
}

```

O/P:

Enter dim of matrix : 3

Enter elements : 1

2

3

4

5

6

7

8

9

Sum = 15

⑤ Delete element from array :

#include ~~<stdio.h>~~

int main()~~{~~

int n, i, pos;

int arr[100];

printf("Enter no. of elts: ");

scanf("%d", &n);

printf("Enter elements: ");

for (i = 0; i < n; i++) {

scanf("%d", &arr[i]);

O/F,

func (" total no of duplicate elements: .d\

Enter

return 0;

Enter

3

4

7

O/P:

The

Enter no. of element: 5

Enter elements: 1

(2) ≤

1

2

inc

3

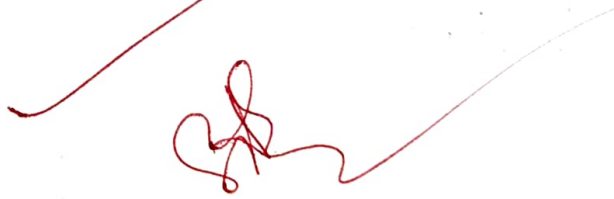
in

4

4

int

Total no. of duplicate elements = 2



LAB - 2

(i) FCFS

```
#include <stdio.h>
```

```
struct process {
```

```
    int pid;
```

```
    int at;
```

```
    int bt;
```

```
    int ct;
```

```
    int tat;
```

```
    int wt;
```

```
    int rt;
```

```
    int st;
```

```
};
```

```
int main() {
```

```
    int n, i, j;
```

```
    float avg-wt = 0;
```

```
    struct process p[20], temp;
```

```
    printf ("Enter no. of processes : ");
```

```
    scanf ("%d", &n);
```

```
    for (i = 0; i < n; i++) {
```

```
        p[i].pid = i+1;
```

```
        printf ("Enter AT & BT for process %d : ",
```

```
                p[i].pid);
```

```
        scanf ("%d %d", &p[i].at, &p[i].bt);
```

```
    }
```

```

for (i = 0; i < n - 1; i++) {
    for (j = i + 1; j < n; j++) {
        if (p[i].at > p[j].at) {
            temp = p[i];
            p[i] = p[j];
            p[j] = temp;
        }
    }
}

```

```

p[0].st = p[0].at;
p[0].ct = p[0].at + p[0].bt;
for (i = 1; i < n; i++) {
    if (p[i].at > p[i-1].ct)
        p[i].st = p[i].at;
    else
        p[i].st = p[i-1].ct;
    p[i].ct = p[i].st + p[i].bt;
}

```

```

for (i = 0; i < n; i++) {
    p[i].tat = p[i].ct - p[i].at;
    p[i].wt = p[i].tat - p[i].bt;
    p[i].rt = p[i].st - p[i].at;
    avg_wt += p[i].wt;
    avg_tat += p[i].tat;
    avg_rt += p[i].rt;
}

```

```
printf ( "\n PID \t AT \t BT \t ST \t CT \t TAT \t  
          \t WT \t RT \n");
```

```
for (i = 0; i < n; i++) {
```

```
    printf ("P %d \t %.2f \t %.2f \t %.2f \t %.2f \t  
            \t %.2f \t %.2f \t %.2f \n");
```

```
    p[i].pid, p[i].at, p[i].bt, p[i].st,
```

```
    p[i].ct, p[i].wt, p[i].tat,  
    p[i].rt);
```

```
}
```

~~return 0;~~

~~printf (" \n average tat = %.2f", avg.tat/n);~~

~~printf (" \n average wt = %.2f", avg.wt/n);~~

~~printf (" \n average rt = %.2f", avg.rt/n);~~

~~return 0;~~

```
}
```

O/P:

enter no. of processes: 3
enter at & bt for P1: 0

enter at and bt for P2: 2

enter at and bt for P3: 4

enter at and bt for P4: 4

5

pid	at	bt	ct	tat	wt	rt
P1	0	7	7	7	0	0
P2	2	4	16	4	5	5
P3	4	4	12	8	3	7
P4	4	5	15	11	6	6

average tat = 8.00

average wt = 2.50

average rt = 2.50

② SJF (Non-preemptive)

#include <stdio.h>

struct process {

int pid; int at; int bt; int ct; int tat;
int wt; int rt; int st; int completed;

};

int main() {

int n, i, time = 0, completed = 0; int idx;

struct process p[20];

printf("Enter no. of processes: ");

scanf("%d", &n);

for (i = 0; i < n; i++) {

p[i].pid = i+1;

printf("Enter AT & BT: ");

scanf("%d", "%d", &p[i].at, &p[i].bt);

p[i].completed = 0;

}

while (completed < n) {

int min_bt = 9999;

idx = -1;

for (i = 0; i < n; i++) {

if (p[i].at <= time & p[i].completed == 0) {

```

i { (p[i].bt < min - bt) {
    min - bt = p[i].bt;
    idx = i;
}
}

if (idx != -1) {
    p[idx].st = time;
    p[idx].ct = p[idx].st + p[idx].bt;
    p[idx].tat = p[idx].ct - p[idx].at;
    p[idx].wt = p[idx].tat - p[idx].bt;
    p[idx].rt = p[idx].st - p[idx].at;
}
}

```

$$\begin{aligned} p[id_x] \cdot st &= hme; \\ p[id_x] \cdot ct &= p[id_x] \cdot st + p[id_x] \cdot bt; \\ p[id_x] \cdot tat &= p[id_x] \cdot ct + p[id_x] \cdot at \\ p[id_x] \cdot wt &= p[id_x] \cdot tat + p[id_x] \cdot bt; \\ p[id_x] \cdot rt &= p[id_x] \cdot st + p[id_x] \cdot at; \end{aligned}$$

```

P[idx]_completed = 1;
completed++;

```

else {

time $t + \tau$;

3

```

pocally ("INP101EAFE16BT1E5F1E6T
1E7A11E6W11E6R111");

```

partially (' p ' . d \ t ' . d \ t ' . d
 \ t ' . d \ t ' . d \ t ' . d \ t ' . d)

3

return 0;

3

018:

Enter no. of processes : 3

labeled at 4 b.t. 0

Enter a & b: 2

4

enter at 4 btr: 4

1

Enter at 9 b t :

pid	at	bt	ct	tat	wt	rt
p1	0	7	7	7	0	0
p2	2	4	12	10	6	6
p3	4	1	8	4	3	3

avg tat = 7.00

avg wt = 3.00

avg rt = 3.00

③ SJF preemptive:

#include <stdio.h>

struct process {

int pid; int at; int bt; int rt-bt;

int ct; int tat; int wt; int rt;

int st; int completed;

};

int main() {

int n, i, time = 0, completed = 0, id;

float avg-tat = 0, avg-wt = 0, avg-rt = 0;

struct process p[20];

printf("Enter no. of processes: ");

scanf("%d", &n);

for (i = 0; i < n; i++) {

p[i].pid = i+1;

printf("Enter at & bt: ");

scanf("%d %d", &p[i].at, &p[i].bt);

p[i].rt-bt = p[i].bt;

p[i].completed = 0;

p[i].st = -1;

}

```

while (completed < n) {
    int min_bt = 9999;
    idx = -1;
    for (i = 0; i < n; i++) {
        if (p[i].at <= time && p[i].completed)
            if (p[i].rt_bt < min_bt &&
                p[i].rt_bt > 0) {
                min_bt = p[i].rt_bt;
                idx = i;
            }
    }
    if (idx != -1) {
        if (p[idx].st == -1) {
            p[idx].st = time;
            p[idx].rt = p[idx].st - p[idx].at;
        }
        p[idx].rt_bt--;
        time++;
        if (p[idx].rt_bt == 0) {
            p[idx].ct = time;
            p[idx].tat = p[idx].ct - p[idx].tat;
            p[idx].wt = p[idx].tat - p[idx].rt;
        }
    }
}

```

```

avg_tat += p[idx].tat;
avg_wt += p[idx].wt;
avg_rt += p[idx].rt;
p[idx].completed = 1;
completed++;
}
}
else {
    time++;
}
}
printf("\nPID\tAT\tRT\tCT\tTAT\tWT\n");
for (i = 0; i < n; i++) {
    printf("%d\t%d\t%d\t%d\t%d\t%d",
        p[i].pid, p[i].at, p[i].bt, p[i].ct,
        p[i].tat, p[i].wt, p[i].rt);
}
avg_tat /= n;
avg_wt /= n;
avg_rt /= n;

```

printf ("In Average TAR = %.2f", avg-tat);
 printf ("In Average WT = %.2f", avg-wt);
 printf ("In Average RT = %.2f", avg-rt);

return 0;

o/p:

Enter no. of processes : 3

Enter at & bt : 0

7

Enter at & bt : 2

4

Enter at & bt : 4

pid	at	bt	ct	tat	wt	rt
p1	0	7	12	12	5	0
p2	2	4	7	5	1	0
p3	4	1	5	1	0	0

avg tat = 6.00

avg wt = 2.00

avg rt = 0.00

conclusion : SJF (preemptive) $\rightarrow$ SRTF $\rightarrow$ is usually faster in terms of avg wt, tat & rt. $\therefore$ more optimal.

LAB-2

PRIORITY SCHEDULING

① Non preemptive:

#include <stdio.h>

```
struct process {
    int pid; int bt; int pr; int wt;
    int ct; int tat; int rt; int at;};
```

int main() {

int n, i, j, time = 0, completed = 0;
 struct process p[10]; temp;
 int visited[10] = {0};

printf ("Enter no. of processes : ");
 scanf ("%d", &n);

for (i = 0; i < n; i++) {

p[i].pid = i+1;
 printf ("Enter at bt pr");
 scanf ("%d %d %d", &p[i].at,
 &p[i].bt, &p[i].pr);

} while (completed < n) {

int idx = -1;
 int highest = 999;


```

for (i=0; i<n; i++) {
    if (p[i].at <= time && visited[i] == 0) {
        if (p[i].pr < highest) {
            highest = p[i].pr;
            idx = i;
        }
    }
}

```

```

if (idx != -1) {
    time += p[idx].bt;
    p[idx].ct = time;
    p[idx].tat = p[idx].ct - p[idx].at;
    p[idx].wt = p[idx].tat - p[idx].bt;
    p[idx].rt = p[idx].wt;

    visited[idx] = 1;
    completed++;
}

```

```

else {
    time++;
}
}

```

```

printf("\nPID\tAT\tBT\tPR\tCT\tTAT\tWT\tRT\n");

```

```

for (i=0; i<n; i++) {
    printf("PID\tAT\tBT\tPR\tCT\tTAT\tWT\tRT\n");
    p[i].pid, p[i].at, p[i].bt, p[i].pr,
    p[i].ct, p[i].tat, p[i].wt,
    p[i].rt);
}

return 0;

```

O/P:

Enter no. of processes : 4

Enter at bt pr 1

2

3

Enter at bt pr 4

3

2

Enter at bt pr 1

4

3

Enter at bt pr 2

4

3

PID	AT	BT	PR	CT	TAT	WT
P1	1	2	3	3	2	0
P2	4	3	2	10	6	3
P3	1	4	3	7	6	2
P4	2	4	3	14	12	8

~~Avg CT = 8.50~~

Avg TAT = 6.50

Avg WT = 3.25

Avg RT = 3.25

② Premptive

```
#include <stdio.h>

struct process {
```

```
    int pid; int bt; int pr; int wt; int ct;
    int tat; int rt; int at; };
```

```
int main() {
```

```
    int n, i, j, time = 0, completed = 0;
```

```
    struct process p[10], temp;
```

```
    int remaining[10];
```

```
    printf("Enter no. of processes: ");
```

```
    scanf("%d", &n);
```

```
    for (i = 0; i < n; i++) {
```

```
        p[i].pid = i + 1;
```

```
        printf("Enter at bt pr");
```

```
        scanf("%d %d %d", &p[i].at,
```

```
                &p[i].bt, &p[i].pr);
```

```
        remaining[i] = p[i].bt;
```

```
    }

    while (completed < n) {
```

```
        int idx = -1;
```

```
        int highest = 999;
```

```

for (i=0; i<n; i++) {
    if (p[i].at <= time && remaining[i] > 0) {
        if (p[i].pr > highest) {
            highest = p[i].pr;
            idx = i;
        }
    }
}

```

```

if (idx != -1) {
    if (remaining[idx] == p[idx].bt) {
        p[idx].rt = time - p[idx].at;
    }
}

```

```

remaining[idx]--;
time++;

```

```

if (remaining[idx] == 0) {
    completed++;
    p[idx].ct = time;
    p[idx].tat = p[idx].ct - p[idx].at;
    p[idx].wt = p[idx].tat - p[idx].bt;
}
}

```

```

else {
    time++;
}
}

```

```

printf("\n PID\tAT\tBT\tPR\tCT\tTAT\tWT\tRT\n");

```

```

for (i=0; i<n; i++) {
    printf("P%d\tAT%d\tBT%d\tPR%d\tCT%d\tTAT%d\tWT%d\tRT%d\n", p[i].pid, p[i].at, p[i].bt, p[i].pr, p[i].ct, p[i].tat, p[i].wt, p[i].rt);
}

```

```

return 0;
}

```


O/P:

Enter no. of processes: 4
Enter at bt pr 1

2
3

Enter at bt pr 4

3
2

Enter at bt pr 1

4
3

Enter at bt pr 2

4
3

PID	AT	BT	PR	CT	TAT	WT	RT
P1	1	2	3	3	2	0	0
P2	4	3	2	7	3	0	0
P3	1	4	3	10	9	5	2
P4	2	4	3	14	12	8	8

Avg TAT = 6.50
Avg WT = 3.25
Avg RT = 2.50

13/3

LAB - 3

28/3/24

ROUND ROBIN
SCHEDULING

#include <stdio.h>

struct process {

int pid;

int at, bt;

int ct, tat, wt, rt;

int remaining_bt;

int first_exec;

int main() {

int n, tq, time = 0, completed = 0;

printf("Enter no. of processes: ");

scanf("%d", &n);

struct process p[n];

for (int i = 0; i < n; i++) {

p[i].pid = i + 1;

printf("Enter AT and BT for P%d:",

p[i].pid);

scanf("%d %d", &p[i].at, &p[i].bt);

p[i].remaining_bt = p[i].bt;

p[i].first_exec = -1;

}

```
printf("Enter time quantum: ");
scanf("%d", & tq);
```

```
while (completed < n) {
    int executed = 0;
```

```
    for (int i = 0; i < n; i++) {
        if (p[i].at <= time & p[i].remaining > 0) {
```

```
            if (p[i].first_exec == -1) {
                p[i].rt = time - p[i].at;
                p[i].first_exec = 1;
```

```
            }
```

```
            if (p[i].remaining_bt > tq) {
                time += tq;
                p[i].remaining_bt = tq;
```

```
            } else {
```

```
                time += p[i].remaining_bt;
```

```
                p[i].remaining_bt = 0;
```

```
                p[i].ct = time;
```

```
                p[i].tat = p[i].ct - p[i].at;
```

```
                p[i].wt = p[i].tat - p[i].bt;
```

```
                completed++;
```

```
            }
            executed = 1;
```

```
        }
```

```
    }
    if (!executed) {
        time++;
```

```
    }
```

```
}
```

```
float avg_tat = 0, avg_wt = 0, avg_rt = 0;
printf("In P10\tA7\tE87\tCT\tTA1\twt\tRT);
```

```
for (int i = 0; i < n; i++) {
```

```
    printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\t");
    p[i].pid, p[i].at, p[i].bt,
    p[i].ct, p[i].tat, p[i].wt, p[i].rt);
```

```
    avg_tat += p[i].tat;
```

```
    avg_wt += p[i].wt;
```

```
    avg_rt += p[i].rt;
```

```
}
```

printf ("In Avg tat = %.2f", avg-tat/n);
 printf ("In Avg wt = %.2f", avg-wt/n);
 printf ("In Avg RT = %.2f", avg-rt/n);

return 0;

3

O/P:

TQ = 1

Enter no. of processes: 3

Enter AT & BT for P1 = 0

5

Enter AT & BT for P2: 1

4

Enter AT & BT for P3: 2

2

Enter Time Quantum: 1

PID	AT	BT	CT	TAT	WT	RT
P1	0	5	11	11	6	0
P2	1	4	10	9	5	0
P3	2	2	6	4	2	0

Avg TAT = 8.00
 Avg WT = 4.33
 Avg RT = 0.00

TQ = 4

Enter no. of processes:

same 1/P

PID	AT	BT	CT	TAT	WT	RT
P1	0	5	11	11	6	0
P2	1	4	8	7	3	3
P3	2	2	10	8	6	1

Avg TAT = 8.67

Avg WT = 5.00

Avg RT = 3.00

overhead

Analysis:

- When TQ is small, response time is better. But more context switching happens which might cause ~~overhead~~.
- When TQ is big, poor response time but fewer context switching.

MULTI LEVEL QUEUING

```
#include <stdio.h>
#include <string.h>
```

```
struct process {
    int pid;
    int at, bt;
    int ct, tat, wt, rt;
    int remaining_bt;
    int first_exec;
    int pri;
    char type[10];
};
```

```
int main() {
    int n, time = 0, completed = 0;
    printf("Enter no. of processes: ");
    scanf("%d", &n);

    struct process p[n];
    for (int i = 0; i < n; i++) {
        p[i].pid = i+1;
        printf("Enter AT, BT & PRI\n\n(1 = system, 2 = user) for p%d:", p[i].pid);
    }
```

```
scanf("%d %d %d", &p[i].at, &p[i].bt, &p[i].pri);
p[i].remaining_bt = p[i].bt;
p[i].first_exec = -1;
```

```
for (int i = 0; i < n-1; i++) {
    if (p[i].at <= p[i+1].at) {
        for (int j = i+1; j < n; j++) {
            if (p[i].at > p[j].at) {
                temp = p[i];
                p[i] = p[j];
                p[j] = temp;
            }
        }
    }
}
```

```
}
}
```

```
while (completed < n) {
    int idx = -1;
    for (int i = 0; i < n; i++) {
        if (p[i].at <= time &&
            p[i].remaining_bt > 0 &&
            p[i].pri == 1) {
                idx = i;
                break;
            }
    }
```



```

printf ("\n Avg TAT = %.2f", avg-tat);
printf ("\n Avg WT = %.2f", avg-wt);
printf ("\n Avg RT = %.2f", avg-rt);

```

return 0;

}

O/p:

Enter no of processes : 3

Enter AT, BT & PRI (1= sys, 2= user) for P1:

2
1

Enter AT, BT & PRI (1= sys, 2= user) for P2:

3
1

Enter AT, BT, & PRI (1= sys, 2= user) for P3:

1
4
2

PID	type	AT	BT	CT	TAT	WT	RT
P1	System	1	2	3	2	0	0
P2	System	1	4	10	9	5	5
P3	user	2	3	6	4	1	1

Avg TAT = 5.00

Avg WT = 2.00

Avg RT = 2.00

~~27/3~~
27/3

modification (include type):

if (p[i].pri == 1)

strcpy (p[i].type, "system");

else

strcpy (p[i].type, "user");

3

Real time (PV) scheduling

Rate-monotonic, EDF, Proportional Scheduling

#include <stdio.h>

#include <math.h>

#define MAX 10

typedef struct {

int id, burst, deadline, period, weight, remaining;

int ct, wt, tat;

} Process;

Process p[MAX];

int n;

float calculateUtilization() {

float u = 0;

for (int i = 0; i < n; i++) {

u += (float)p[i].burst / p[i].period;

}

return u;

}

void edf() {

printf("\n -- EDF scheduling -- \n");

float u = calculateUtilization();

printf("CPU utilization : %.2f\n", u);

if (u <= 1)

printf("Schedulable (u <= 1)");

else {

printf("Not schedulable");

return;

}

for (int i = 0; i < n - 1; i++) {

for (int j = i + 1; j < n; j++) {

if (p[i].deadline > p[j].deadline) {

Process temp = p[i];

p[i] = p[j];

p[j] = temp;

}

}

}

int time = 0;

for (int i = 0; i < n; i++) {

time += p[i].burst;

p[i].ct = time;

p[i].tat = p[i].ct;

p[i].wt = p[i].tat - p[i].burst;

}

printf("\n ID BF Deadline CT WT TAT\n");

for (int i = 0; i < n; i++) {

printf("%d\t%d\t%d\t%d\t%d\t",

p[i].id, p[i].burst, p[i].deadline, p[i].ct,

p[i].wt, p[i].tat);

```

}
}
void rms() {
    printf("\n --- RMS --- \n");
    float u = calculateUtilization();
    printf("CPU utilization: %.2f\n", u);

    float bound = n * (pow(2, (10/n)) - 1);
    printf("RM bound: %.4f\n", bound);

    if (u <= bound)
        printf("Schedulable");
    else {
        printf("Not schedulable");
        return;
    }
}

```

```

}
for (int i=0; i<n-1; i++) {
    for (int j=i+1; j<n; j++) {
        if (p[i].period > p[j].period) {
            Process temp = p[i];
            p[i] = p[j];
            p[j] = temp;
        }
    }
}
}

```

```

int time = 0;
for (int i=0; i<n; i++) {
    time += p[i].burst;
    p[i].ct = time;
    p[i].tat = p[i].ct;
    p[i].wt = p[i].tat - p[i].burst;
}

```

```

printf("\n ID: BF Period (T) WT TAT\n");
for (int i=0; i<n; i++) {
    printf("%d\t%d\t%d\t%d\t%d\t%d\t",
        p[i].id, p[i].burst, p[i].period,
        p[i].ct, p[i].wt, p[i].tat);
}

```

```

}
void proportional() {
    printf("\n --- Proportional scheduling --- \n");
    int total_weight = 0;
    for (int i=0; i<n; i++) {
        total_weight += p[i].weight;
        p[i].remaining = p[i].burst;
    }
    int time = 0; completed = 0;
}

```

```

while (completed < n) {
    for (int i = 0; i < n; i++) {
        int share = p[i].weight;
        while (share -- && p[i].remaining > 0) {
            time++;
            p[i].remaining--;

            if (p[i].remaining == 0) {
                p[i].ct = time;
                p[i].tat = p[i].ct;
                p[i].wt = p[i].tat - p[i].burst;
                completed++;
                break;
            }
        }
    }
}

```

```

printf("\nID BF weight CT WT TAT\n");
for (int i = 0; i < n; i++) {
    printf("%d\t%d\t%d\t%d\t%d\n",
        p[i].id, p[i].burst, p[i].weight,
        p[i].ct, p[i].wt, p[i].tat);
}

```

```

int main() {
    printf("Enter no. of processes: ");
    scanf("%d", &n);
    printf("\nEnter process details: \n");
}

```

```

for (int i = 0; i < n; i++) {
    printf("\nProcess %d: \n", i);
    p[i].id = i;
    printf("BT: ");
    scanf("%d", &p[i].burst);
    printf("Deadline for ADF: ");
    scanf("%d", &p[i].deadline);
    printf("Period for RMS: ");
    scanf("%d", &p[i].period);
    printf("Weight: ");
    scanf("%d", &p[i].weight);
}

```

```

edf();
rms();
proportional();

```

return 0;

O/P:

Enter no. of processes: 3

Enter process details:

Process 0:

Burst time: 3

Deadline for EDF = 7

Period for RMS = 20

Weight: 2

Process 1:

Burst Time: 2

Deadline: 4

Period: 5

Weight = 3

Process 2:

BT: 2

Deadline: 8

Period: 10

Weight: 5

--- EDF ---
CPU utilization: 0.75
Schedulable ($U < 1$)

ID	BF	Deadline	CT	WT	TAT
1	2	4	2	0	2
0	3	7	5	2	5
2	2	8	7	5	7

-- RM --

RM bound = 0.7798
Schedulable ($U < \text{RM}$)

ID	BF	Period	CT	WT	TAT
1	2	5	2	0	2
2	2	10	4	2	4
0	3	20	7	4	7

-- Proportional --

ID	BF	weight	CT	WT	TAT
1	2	3	2	0	2
2	2	5	4	2	4
0	3	2	7	4	7

Signature

WEEK - 6

Bounded Buffer

wait

Trace

Buffer size : 3

Buffer = [_ , _ , _]

in = 0 out = 0

empty = 3 full = 0, mutex = 1

Set 24/4.

Step #	Operation	Buffer State	in	out	empty	full	mutex
1	Produce 1	1 _ _	1	0	2	1	1
2	Consume 1	_ _ _	1	1	3	0	1
3	Produce 2	_ 2 _	2	1	2	1	1
4	Produce 3	_ 2 3	0	1	1	2	1
5	Produce 2	2 2 3	1	1	0	3	1
6	Produce 3	<u>Buffer full</u>					
7	Consume 2	2 _ 3	1	2	1	2	1

Dining Philosophers

step	Philosopher	Action	Status
1	P ₀	Thinking	(F, F, F, F, F)
2	P ₀	Picks L fork	(0 F F F F)
3	P ₀	Picks R	(0 0 F F F)
4	P ₀	Eating	(0 0 F F F)
5	P ₀	Putts fork 0 & 1	(P F F F F)
6	P ₁	Thinking	(F F F F F)
7	P ₁	Picks L fork 1	(F 1 F F F)
8	P ₁	Picks R Fork 2	(F 1 1 F F)
9	P ₁	Eating	(F 1 1 F F)
10	P ₁	Putts fork 1, 2	(F F F F F)
11	P ₃	Thinking	(F F F 3 F)
12	P ₃	Picks L 3	(F F F 3 3)
13	P ₃	Picks R 4	(F F F 3 3)
14	P ₃	Eat	(F F F 3 3)
15	P ₃	put back	(F F F F F)
16	P ₂	Picks L 2	(F F 2 F F)
17	P ₂	Pick R 3	(F F 2 2 F)
18	P ₂	Eat	(F F 2 2 F)
19	put back P ₂	put back	(F F F F F)

Dining Philosophers

```
#include <stdio.h>
#include <semaphore.h>
#include <unistd.h>
#define N 5
sem_t forks[N]

void * philosopher (void * num) {
    int id = (int * num);
    while (1) {
        printf ("philosopher %d is thinking\n", id);
        sleep (1);
        if (id % 2 == 0) {
            sem_wait (& forks[id]);
            printf ("philosopher %d picks up left fork from %d\n", id, id);
            sem_wait (& forks[(id+1)%N]);
            sem_wait (& forks[id]);
            printf ("philosopher %d picked up right fork\n", id, id);
            (id+1) % N;
        }
    }
}
```

```
} else {
    sem_wait (& forks[id]),
    printf ("philosopher %d picked left");
}

printf ("philosopher %d is eating");
sleep (1);
sem_post (& forks[id]);
sem_post (& forks[(id+1)%N]);
sem_post (& room);
}
```

```
}
int main () {
    pthread_t * phil[N];
    int id[N];
    for (int i = 0; i < N; i++) {
        sem_init (& forks[i]);
    }
    for (int i = 0; i < N; i++) {
        pthread_join (phil[N]);
    }
    return 0;
}
```

Producer - consumer

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>

#define BUFFER_SIZE 5
#define MAX 15

int buffer [BUFFER_SIZE];
int in = 0; out = 0;
sem_t empty, full, mutex;

void * producer (void * arg) {
    for (int i = 0; i < max; i++) {
        sem_wait (&empty);
        sem_wait (&mutex);
        buffer[in] = i;
        printf ("produced i.d at buffer i.d")
            is in );
        in = (in + 1) % BUFFER_SIZE;
        sem_post (&mutex);
        sem_post (&full);
        sleep (1);
    }
    return NULL;
}
```

```
void * consumer (void * arg) {
    for (int i = 0; i < max; i++) {
        sem_wait (&full);
        sem_wait (&mutex);
        int item = buffer[out];
        printf ("consumer i.d from
            buffer [i.d]n", item, out);
        out = (out + 1) % BUFFER_SIZE;
        sem_post (&mutex);
        sem_post (&empty);
        sleep (1);
    }
    return NULL;
}
```

```
int main() {
    pthread_t, p, c;
    sem_init (&empty, 0, BUFFER_SIZE);
    sem_init (&full, 0, 0);
    sem_init (&mutex, 0, 1);
    pthread_create (&p, NULL, produce, NULL);
    pthread_create (&c, NULL, consume, NULL);
    pthread_join (p, NULL);
    pthread_join (c, NULL);
}
```

```
sem_destroy (&empty);
sem_destroy (&full);
sem_destroy (&mutex);
```

```
} return 0;
```

O/P

Produced : 0 at buffer [0]

Consumer : 0 from buffer [0]

Produced : 1 at buffer [1]

Consumer : 1 from buffer [1]

WEEK - 7

8/5/26

① BANKER'S ALGORITHM

```
#include <stdio.h>
```

```
int main () {
```

```
int n, m, i, j, k;
```

```
printf ("Enter no. of processes: ");
```

```
scanf ("%d", &n);
```

```
printf ("Enter no. of resources: ");
```

```
scanf ("%d", &m);
```

```
int allocation[n][m], max[n][m],
```

```
need[n][m];
```

```
int available[m];
```

```
printf ("Enter allocation matrix: \n");
```

```
for (i=0; i<n; i++) {
```

```
for (j=0; j<m; j++) {
```

```
scanf ("%d", &allocation[i][j]);
```

```
}
```

```
}
```

```
printf ("Enter maximum demand matrix");
for (i=0; i<n; i++) {
```

```
    for (j=0; j<m; j++) {
        scanf ("%d", & allocation max [i][j]);
    }
}
```

```
3 }
```

```
printf ("Enter Available Resources: \n");
for (i=0; i<m; i++) {
    scanf ("%d", & available [i]);
}
```

```
for (i=0; i<n; i++) {
    for (j=0; j<m; j++) {
        need [i][j] = max [i][j] -
            allocation [i][j];
    }
}
```

```
int kwork [m], finish [n], safeSequence [n];
```

```
for (i=0; i<m; i++) {
    work [i] = available [i];
}
```

```
for (i=0; i<n; i++)
    finish [i] = 0;
```

```
int count = 0;
```

```
while (count < n) {
    int found = 0;
```

```
    for (i=0; i<n; i++) {
        if (finish [i] == 0) {
```

```
            for (j=0; j<m; j++) {
                if (need [i][j] > work [j])
                    break;
            }
        }
```

```
        if (j == m) {
            for (k=0; k<m; k++)
                work [k] += allocation [i][k];
        }
```

```
        safeSequence [count++] = i;
        finish [i] = 1;
        found = 1;
    }
```

```
    if (found == 0) {
        printf ("System is UNSAFE");
        return 0;
    }
}
```

```
3 }
```


printf ("In System is in SAFE state :");
printf ("Safe Sequence :");

for (i=0, i<n, i++) {
 printf ("P%d", safeSequence[i]);

}

return 0;

}

O/P:

Enter no of processes : 5

Enter no of resource types : 3

Enter allocation Matrix :

0 1 0
2 0 0
3 0 2
2 1 1
0 0 2

Enter maximum matrix :

7 5 3
3 2 2
4 0 2
2 2 2
4 3 3

Enter available resource :

3 3 2

System is in SAFE ~~Resource~~ state

Safe Sequence : P1 P3 P4 P0 P2

(2) Deadlock Detection

```
#include <stdio.h>
```

```
int main() {  
    int n, m, i, j, k;  
    printf("Enter no. of processes: ");  
    scanf("%d", &n);  
  
    printf("Enter no. of resource types: ");  
    scanf("%d", &m);  
  
    int allocation[n][m], request[n][m];  
    int available[m];  
  
    printf("Enter Allocation Matrix: \n");  
    for (i = 0; i < n; i++) {  
        for (j = 0; j < m; j++) {  
            scanf("%d", &allocation[i][j]);  
        }  
    }  
  
    printf("Enter request Matrix: \n");  
    for (i = 0; i < n; i++) {  
        for (j = 0; j < m; j++) {  
            scanf("%d", &request[i][j]);  
        }  
    }  
}
```

```
printf("Enter available resources: \n");  
for (i = 0; i < m; i++) {  
    scanf("%d", &available[i]);  
}
```

```
int work[m], finish[n];  
for (i = 0; i < m; i++) {  
    work[i] = available[i];  
}
```

```
for (i = 0; i < n; i++) {  
    int flag = 0;  
    for (j = 0; j < m; j++) {  
        if (allocation[i][j] != 0) {  
            flag = 1;  
            break;  
        }  
    }  
    if (flag == 1)  
        finish[i] = 0;  
    else  
        finish[i] = 1;  
}
```

```
for (i = 0; i < n; i++) {  
    if (finish[i] == 0) {  
        for (j = 0; j < m; j++) {  
            work[j] = work[j] + allocation[i][j];  
        }  
    }  
}
```

```

    if (request[i][j] > work[j])
        break;
    }
    if (j == m) {
        for (k = 0; k < m; k++) {
            work[k] += allocation[i][k];
        }
        finish[i] = 1;
        i = -1;
    }
}
}

```

```

int deadlock = 0;
for (i = 0; i < n; i++) {
    if (finish[i] == 0) {
        printf("Process %d is deadlocked", i);
        deadlock = 1;
    }
}

```

```

if (deadlock == 0) {
    printf("No deadlock Detected\n");
}
return 0;
}

```

no. of processes : 5
no. of resource types : 3
allocation matrix:

```

0 1 0
2 0 0
3 0 3
2 1 1
0 0 2

```

Req matrix:

```

0 0 1
2 0 2
1 0 0
1 0 0
0 0 2

```

available resources:

```

0 0 0

```

Process P₀ is deadlocked
Process P₁ is deadlocked
Process P₂ is deadlocked
Process P₄ is deadlocked

5/11

Contiguous Memory Allocation Techniques

```

#include <stdio.h>

void firstFit ( int b[], int m, int p[], int n ) {
    int temp[m], a[n], used[m];

    for ( int i = 0; i < m; i++ )
        temp[i] = b[i];

    for ( int i = 0; i < n; i++ )
        a[i] = -1;

    for ( int i = 0; i < n; i++ )
        for ( int j = 0; j < m; j++ ) {
            if ( usedtemp[j] >= p[i] ) {
                a[i] = j;
                usedtemp[j] = p[i] + 1;
                break;
            }
        }

    printf ( "\n --- First Fit --- \n" );
    printf ( " Process No. \t Process Size \t Block No. \n" );

    for ( int i = 0; i < n; i++ ) {
        printf ( "%d \t \t %d \t \t ", i+1, p[i] );
        if ( a[i] != -1 )
            printf ( "%d \n", a[i] + 1 );
        else
            printf ( "Not allocated \n" );
    }
}

```

```

void bestFit( int b[], int m, int p[], int n) {
    int temp[m], a[n], used[m];
    for (int i=0; i<m; i++)
        temp[i] = b[i];

```

```

    for (int i=0; i<n; i++)
        a[i] = -1;

```

```

    for (int i=0; i<n; i++) {
        int best = -1;

```

```

        for (int j=0; j<m; j++) {
            if (usedtemp[j] >= p[i]) {
                if (best == -1 || usedtemp[j] < usedtemp[best])
                    best = j;
            }

```

```

        }
        if (best != -1) {
            a[i] = best;
            usedtemp[best] = p[i];
        }
    }

```

```

    printf("\n --- Best Fit --- \n");

```

```

    printf("Process No \t Process Size \t Block no. \n");

```

```

    for (int i=0; i<n; i++) {
        printf("%d \t %d \t", i+1, p[i]);
        if (a[i] != -1)
            printf("%d \n", a[i]+1);
        else
            printf("Not allocated \n");
    }
}

```

```
void worstFit(int b[], int m, int p[], int n){
```

```
int tempused[m], a[n];
```

```
for (int i=0; i<m; i++){  
temp[i] = b[i];
```

```
for (int i=0; i<n; i++){  
a[i] = -1;
```

```
for (int i=0; i<n; i++){  
int worst = -1;
```

```
for (int j=0; j<m; j++){
```

```
if (tempused[j] >= p[i]) {
```

```
if (worst == -1 || tempused[j] >  
tempused[worst])
```

```
worst = j;
```

```
}
```

```
if (worst != -1) {
```

```
a[i] = worst;
```

```
temp[worst] = p[i];
```

```
}
```

```
printf("\n --- Worst Fit --- \n");
```

```
printf("Process No. \t Process Size \t Block No. \n");
```

```
for (int i=0; i<n; i++){
```

```
printf("%d \t \t %d \t \t", i+1, p[i]);
```

```
if (a[i] != -1)
```

```
printf("%d \n", a[i]+1);
```

```
else  
printf("Not allocated \n");
```

```
}
```

```
}
```



```

int main() {
    int m, n;
    printf("Enter no. of memory blocks: ");
    scanf("%d", &m);
    int b[m];
    printf("Enter sizes of i.d mem blocks: \n, m);
    for (int i = 0; i < m; i++)
        scanf("%d", &b[i]);

    printf("Enter no. of processes: ");
    scanf("%d", &n);
    int p[n];
    printf("Enter sizes of i.d processes \n, n);
    for (int i = 0; i < n; i++)
        scanf("%d", &p[i]);

    firstFit(b, m, p, n);
    bestFit(b, m, p, n);
    worstFit(b, m, p, n);

    return 0;
}

```

O/E

Enter sizes of 5 memory blocks:
 100 500 200 300 600

Enter no. of processes: 4

Enter sizes of 4 processes:
 212 417 112 426

--- First Fit ---

Process No.	Process size	Block No.
1	212	2
2	417	5
3	112	3
4	426	Not allocated

--- Best Fit ---

Process No.	Process size	Block No.
1	212	4
2	417	2
3	112	3
4	426	5

--- Worst Fit ---

Process No.	Process size	Block No.
1	212	5
2	417	2
3	112	4
4	426	Not allocated

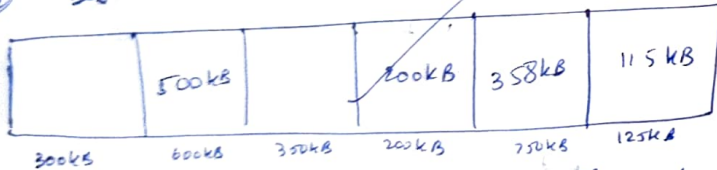
Q. Given six memory partitions of
 300kB, 600kB, 350kB, 200kB, 750kB
 and 125kB (in order), how would
 best fit, worst fit & first fit algorithms
 place the processes of sizes:
 115kB, 500kB, 350kB, 200kB, 375kB (in order)

sol:
 ① First Fit:



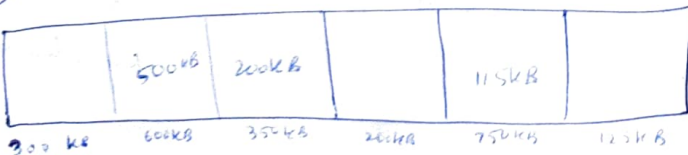
Process with 375kB cannot be allocated.

② Best Fit:



Process with 375kB cannot be allocated.

③ Worst Fit:



Process with sizes 358kB & 375kB cannot be allocated.

LAB-9

22/5/26

PAGEREPLACEMENT

1) FIFO 2) LRU 3) Optimal

#include <stdio.h>

```

int main() {
    int ref[20], n, f;
    int i, j, k;

    printf("Enter no. of pages");
    scanf("%d", &n);

    printf("Enter reference string");
    for (i = 0; i < n; i++)
        scanf("%d", &ref[i]);

    int fifo[10], fifoFault = 0, index = 0, found;

    for (i = 0; i < f; i++)
        fifo[i] = -1;

    for (i = 0; i < n; i++) {
        found = 0;
        for (j = 0; j < f; j++) {
            if (fifo[j] == ref[i]) {
                found = 1;
                break;
            }
        }

        if (found == 0) {
            fifo[index] = ref[i];
            index = (index + 1) % f;
            fifoFault++;
        }
    }
}

```

int brn[10], time[10], brnFault = 0;

int pos, least;

for (i = 0; i < f; i++) {

brn[i] = -1;

time[i] = -1;

}

for (i = 0; i < n; i++) {

found = 0;

for (j = 0; j < f; j++) {

if (brn[j] == req[i]) {

found = 1;

time[j] = 1;

break;

}

if (found == 0) {

least = time[0];

post = 0;

for (j = 1; j < f; j++) {

if (time[j] < least) {

least = time[j];

post = j;

}

brn[pos] = req[i];

time[pos] = i;

brnFault++;

}

}

int opt[10], optFault = 0;

int farthest;

for (i = 0; i < f; i++) {

opt[i] = -1;

for (i = 0; i < n; i++) {

found = 0;

for (j = 0; j < f; j++) {

if (opt[j] == req[i]) {

found = 1;

break;

}

if (found == 0) {

for (j = 0; j < f; j++) {

if (opt[j] == -1) {

post = j;

break;

}

if (j == f) {

farthest = -1;

for (j = 0; j < f; j++) {

int next = -1;

for (k = i+1; k < n; k++) {

if (next == -1) {

post = j;

break;

}

```

if (next > farthest) {
    farthest = next;
    next = j;
}
}
}

```

```

}
opt[page] = ref[i];
optFault++;
}
}

```

```

printf("\n FIFO Page Faults = %d", fifoFault);
printf("\n LRU Page Faults = %d", LRUFault);
printf("\n Optimal Page Faults = %d", optFault);

```

```

return 0;
}

```

O/e;

Enter number of pages : 12

Enter page reference string : 1 2 3 4 1 2 5 1 2 3 4 5

Enter no. of frames : 3

FIFO Page Faults = 9

LRU Page Faults = 10

Optimal Page Faults = 7

Q62
22/5/26